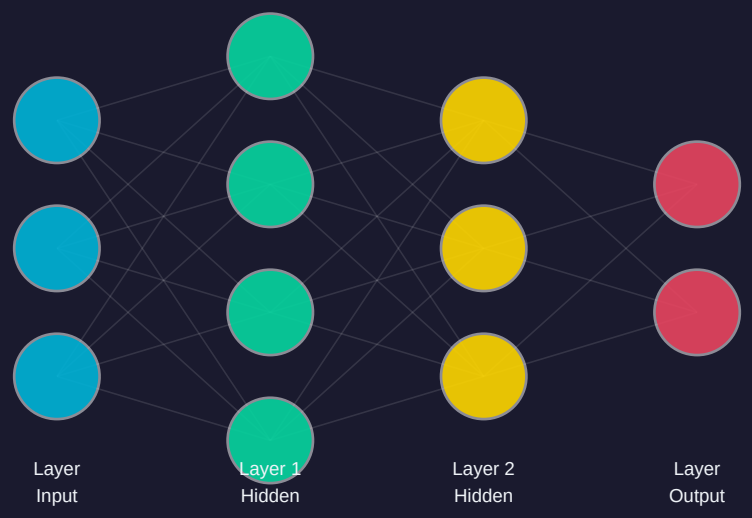
CHAPTER 3

Neural Networks

Complete Study Guide

Conceptual Notes • Mathematics • Architecture • Training • Applications



Sections Covered

Concepts | Math | Activation Functions | Architecture | Training | Applications | Quiz | Glossary

Section 1: Conceptual Notes

What is a Neural Network?

A **Neural Network** is a computational model loosely inspired by the human brain. It consists of layers of interconnected nodes (neurons) that process data by passing signals through weighted connections. Neural networks learn patterns by adjusting these weights during a training process, enabling them to perform tasks such as image recognition, language translation, and game-playing—often with human-level accuracy.

At its core, a neural network transforms an input (e.g., pixels of an image) through a series of mathematical operations into an output (e.g., the label 'cat'). The power lies in *learning*: given enough examples, the network discovers the right internal representations automatically.

Biological vs Artificial Neuron

Feature	Biological Neuron	Artificial Neuron
Signal Input	Dendrites receive electrochemical signals	Inputs x1, x2, … xn (numbers)
Processing	Cell body integrates signals	Weighted sum: $z = \Sigma(w_i \cdot x_i) + b$
Output	Axon fires if threshold exceeded	Activation function: $a = f(z)$
Learning	Synaptic plasticity (Hebbian)	Weight updates via backpropagation
Speed	~200 Hz, very slow	Millions of operations/second
Parallelism	~86 billion neurons in parallel	Thousands of units (GPU parallelism)

Deep Learning

Deep Learning is a subfield of machine learning that uses neural networks with *many hidden layers* (hence 'deep'). Each layer learns increasingly abstract representations of the data:

- Layer 1 (edges & colours) → Layer 2 (shapes) → Layer 3 (parts) → Layer 4 (objects)

This hierarchical feature extraction is what makes deep networks extraordinarily powerful for raw data such as images, audio, and text—where hand-crafted features are impractical.

> *Key insight: More layers = more abstract representations, but also more data and compute required.*

Section 2: Neuron Mathematics

Weighted Sum (Pre-activation)

Every neuron computes a weighted combination of its inputs plus a bias term. Think of **weights** as *importance knobs*—a high weight means that input matters a lot; a low weight means it barely matters.

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_n \cdot x_n + b = W \cdot X + b$$

- **xi** — the i-th input feature (e.g., pixel brightness)
- **wi** — the weight for that input (learned during training)
- **b** — the bias, a learnable offset that shifts the activation threshold
- **z** — the pre-activation value (raw weighted sum)

Activation Function

The activation function introduces **non-linearity**. Without it, stacking layers would still produce only linear transformations—no more powerful than a single layer. Non-linearity allows the network to learn complex, curved decision boundaries.

a = f(z) where f is the chosen activation function

Loss Function

The loss function measures **how wrong** the network's predictions are. During training we minimise this number. Two common choices:

MSE = (1/n) · Σ (yi - y_hat_i)² [Regression]

Cross-Entropy = -Σ yi · log(y_hat_i) [Classification]

- **yi** — true label
- **y_hat_i** — network's prediction

Intuition: MSE punishes large errors quadratically. Cross-entropy strongly penalises confident wrong answers—pushing the network to be well-calibrated.

Section 3: Activation Functions

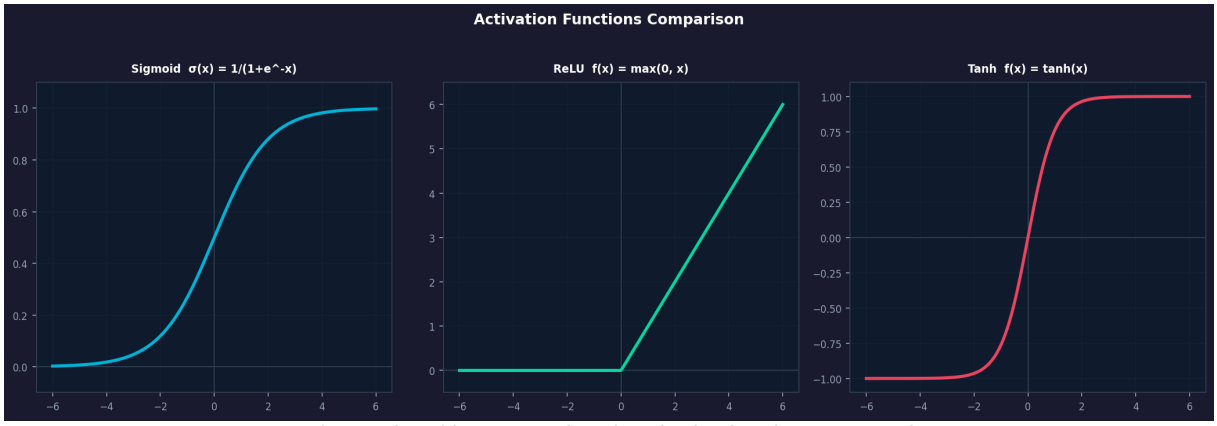


Figure: Sigmoid, ReLU, and Tanh activation functions compared

Function	Formula	Output Range	Best Used For	Key Weakness
Sigmoid	σ(x) = 1/(1+e^-x)	(0, 1)	Binary classification output	Vanishing gradients for large |x|
ReLU	max(0, x)	[0, ∞)	Hidden layers (default choice)	Dead neurons (always 0 for x<0)

Tanh	tanh(x)	(-1, 1)	Hidden layers, RNNs	Vanishing gradients (milder than Sigmoid)

Sigmoid in Detail

Sigmoid squashes any real number into (0, 1)—perfect for outputting probabilities. However, for very large or very small inputs, its gradient becomes near-zero, slowing or halting learning in deep networks (the *vanishing gradient problem*).

ReLU in Detail

Rectified Linear Unit is computationally cheap and avoids vanishing gradients for positive values. It is the default choice for hidden layers in modern networks. Variants like Leaky ReLU and ELU address the 'dead neuron' issue by allowing a small negative slope for $x < 0$.

Tanh in Detail

Tanh is zero-centred (output in –1 to 1), which often leads to faster convergence than Sigmoid. It is commonly used in recurrent neural networks (RNNs) and for hidden layers when data is naturally centred around zero.

Section 4: Network Architecture

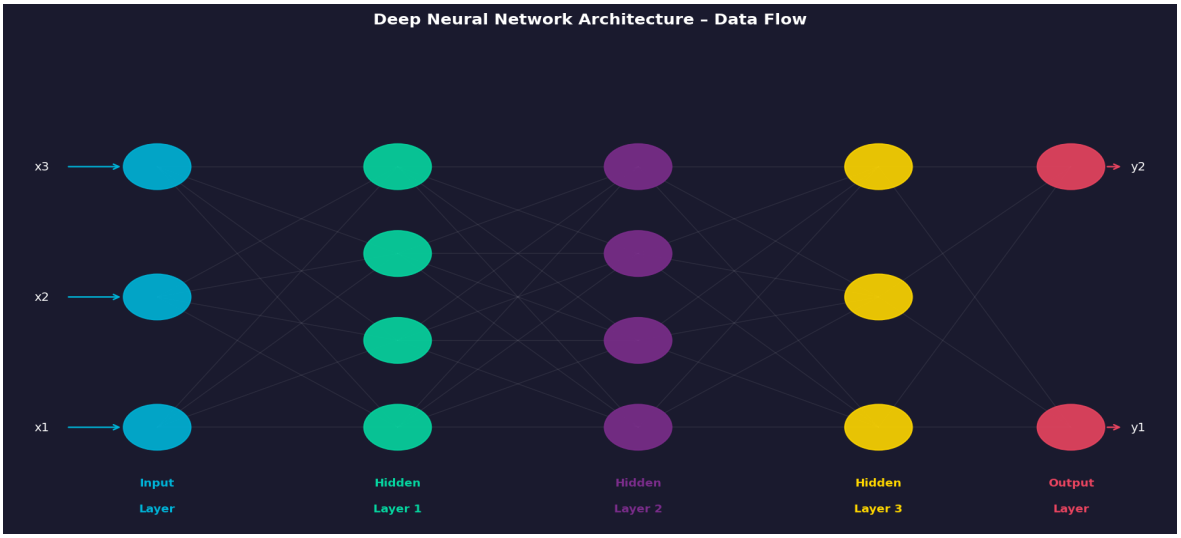


Figure: Deep neural network with three hidden layers showing data flow

Input Layer

The input layer receives raw data. Each node corresponds to one feature (e.g., one pixel in an image, one word embedding dimension in NLP). **No computation happens here**—the layer simply passes data forward. The number of input nodes equals the dimensionality of your feature vector.

Hidden Layers

Hidden layers perform the actual **feature extraction and transformation**. Each hidden neuron computes $z = W \cdot x + b$, then $a = f(z)$. With multiple hidden layers, the network learns hierarchical representations: early layers detect simple features, later layers combine them into complex concepts. The number of hidden layers and neurons per layer are *hyperparameters* chosen by the designer.

Output Layer

The output layer produces the network's final prediction. Its design depends on the task:

- **Binary classification** — 1 neuron, Sigmoid activation → probability in (0,1)
- **Multi-class classification** — k neurons (one per class), Softmax activation → probabilities sum to 1
- **Regression** — 1 neuron, no activation (or linear) → real-valued output

Data Flow

Data flows **forward** from input → hidden layers → output (forward propagation). During training, error signals flow **backward** from output → hidden layers → input (backpropagation), updating weights at each step. After training, only forward propagation is needed for inference.

Section 5: Training Process

Forward Propagation

Forward propagation is the process of computing predictions. Given an input X, we sequentially compute activations through each layer:

$$a[0] = X \rightarrow z[l] = W[l] \cdot a[l-1] + b[l] \rightarrow a[l] = f(z[l]) \rightarrow \ldots \rightarrow y_hat$$

The superscript [l] denotes layer l. The final output y_hat is compared to the true label y using the loss function to compute a scalar error.

Backpropagation

Backpropagation is the algorithm for computing *gradients*—how much each weight contributed to the error. It applies the **chain rule of calculus** to propagate the loss gradient backwards through the network:

$$\partial L/\partial W[l] = \partial L/\partial a[l] \cdot \partial a[l]/\partial z[l] \cdot \partial z[l]/\partial W[l]$$

Intuitively: if increasing a weight made the loss go up, we decrease it; if it made the loss go down, we increase it. Backprop computes all these 'blame' signals efficiently in a single backward pass—much faster than computing each gradient separately.

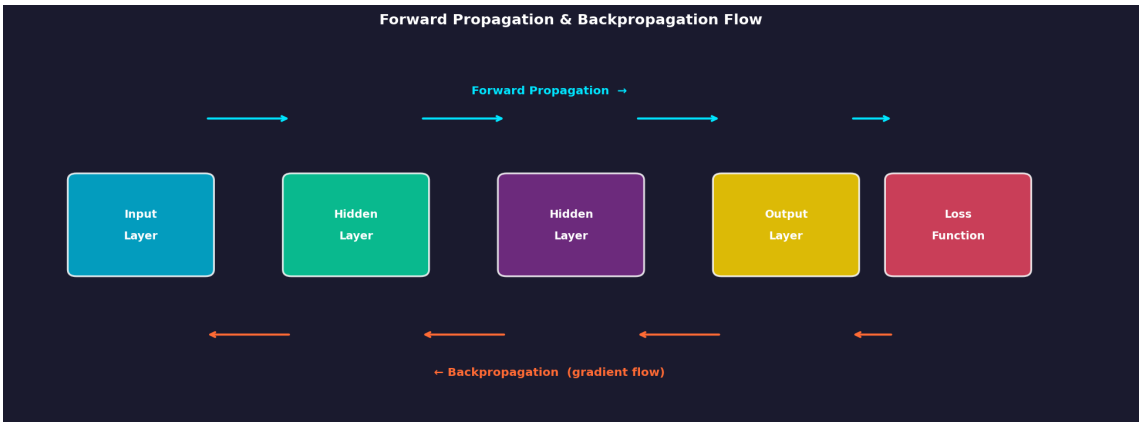


Figure: Forward propagation (blue) and backpropagation error flow (orange)

Gradient Descent

Once we have gradients, we update weights using gradient descent. The learning rate η controls the step size:

$$W \leftarrow W - \eta \cdot \partial L / \partial W$$

Figure: Gradient descent navigating a loss landscape toward the minimum

- **Vanilla Gradient Descent** — uses all training data per update (slow but stable)
- **Stochastic Gradient Descent (SGD)** — uses one sample per update (noisy but fast)
- **Mini-batch GD** — uses small batches (best of both worlds, standard in practice)
- **Adam, RMSProp** — adaptive optimisers with per-parameter learning rates (widely used)

Section 6: Visual Reference Diagrams

Diagram 1 – Single Neuron (Perceptron Model)

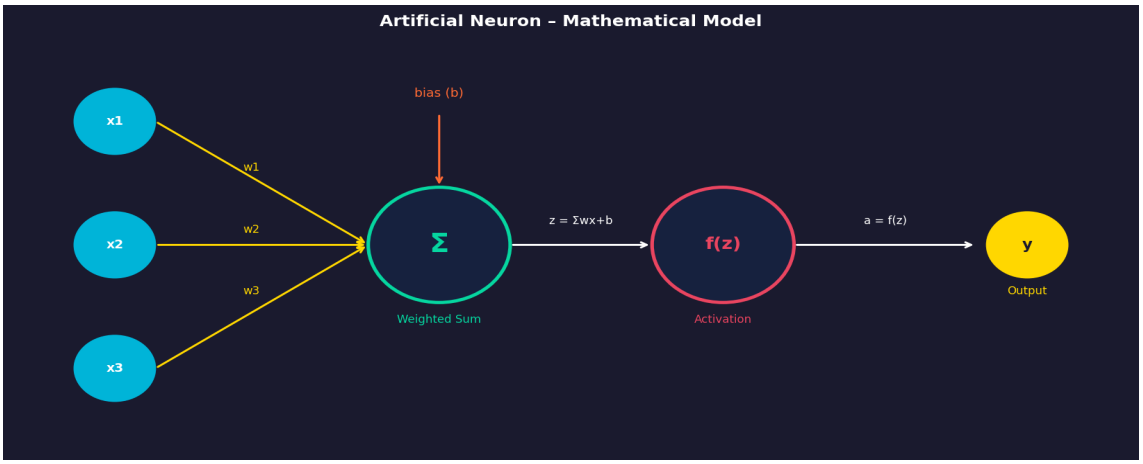


Figure: A single artificial neuron with 3 inputs, learned weights, bias, summation (Σ), activation f(z), and output.

Section 7: Real-World Applications

Domain	Application	Network Type	How It Works
Computer Vision	Face Recognition	CNN (Convolutional Neural Network)	Convolutional layers extract local features (edges, textures, ey
Natural Language Processing	Machine Translation Sentiment Analysis Chatbots	Transformer / LSTM / RNN	Attention mechanisms capture long-range word dependencies.
Autonomous Systems	Self-driving Cars Robot Navigation Drone Control	CNN + Reinforcement Learning	Cameras feed into CNNs to detect lanes, pedestrians, obstacle
Healthcare	Medical Imaging Drug Discovery	CNN / Graph Neural Networks	CNNs classify tumours in X-rays / MRIs with radiologist-level a
Finance	Fraud Detection Algorithmic Trading	LSTM / Autoencoder	LSTMs model time-series spending patterns; anomalies flag fra

---

Section 8: Practice Quiz

Answer all questions. Answers with explanations follow in Section 9.

Q1. What is the primary purpose of an activation function in a neural network?

A) To initialise weights randomly

B) To introduce non-linearity into the network

C) To compute the loss between prediction and label

D) To normalise the input data

Q2. Which formula correctly represents the weighted sum in a single neuron?

A) $a = f(W \cdot X + b)$

B) $z = W \cdot X + b$

C) $L = \Sigma(yi - y_hat)^2$

D) $\sigma(x) = 1 / (1 + e^x)$

Q3. What does 'backpropagation' compute?

A) The forward pass predictions

B) The loss on the validation set

C) Gradients of the loss with respect to each weight

D) The softmax probabilities

Q4. Which activation function outputs values strictly between 0 and 1?

A) ReLU

B) Tanh

C) Sigmoid

D) Leaky ReLU

Q5. What is the role of the bias term (b) in a neuron?

A) To scale the input features

B) To shift the activation threshold independently of inputs

C) To regularise the weights

D) To compute the gradient

Q6. In a multi-class classification network with 5 classes, how many output neurons are needed?

A) 1

B) 2

C) 5

D) Depends on the hidden layers

Q7. What is the 'vanishing gradient' problem?

A) Weights become too large during training

B) Gradients become near-zero in early layers, halting learning

C) The loss function returns negative values

D) Output neurons produce values greater than 1

Q8. Which optimiser uses adaptive per-parameter learning rates?

A) Vanilla SGD

B) Gradient Descent

C) Adam

D) Forward Propagation

Q9. What type of neural network is most commonly used for image recognition?

A) RNN

B) LSTM

C) CNN (Convolutional Neural Network)

D) Autoencoder

Q10. What does the learning rate (η) control?

A) Number of training epochs

B) Size of the weight update step during gradient descent

C) Number of hidden layers

D) The activation function used

Q11. Which loss function is typically used for binary classification?

A) Mean Squared Error (MSE)

B) Cross-Entropy

C) Huber Loss

D) L1 Loss

Q12. What happens during a forward pass?

A) Weights are updated based on gradients

B) Input data flows through the network to produce a prediction

C) The loss is backpropagated

D) The model is initialised

Q13. What is an 'epoch' in neural network training?

A) A single weight update

B) One complete pass through the entire training dataset

C) The number of hidden layers

D) The learning rate schedule

Q14. ReLU(–5) equals:

A) –5

B) 5

C) 0

D) 0.5

Q15. Deep learning differs from traditional machine learning primarily because:

A) It requires less data

B) It uses multiple abstraction layers to learn hierarchical representations automatically

C) It does not use gradient descent

D) It can only be applied to image data

Section 9: Answer Key & Explanations

Q1. Answer: B

Activation functions like ReLU and Sigmoid introduce non-linearity. Without them, any stack of linear layers collapses to a single linear transformation, severely limiting the model's expressive power.

Q2. Answer: B

$z = W \cdot X + b$ is the pre-activation (weighted sum). Option A is the activation output $a = f(z)$, C is MSE loss, and D is sigmoid with an incorrect sign.

Q3. Answer: C

Backpropagation applies the chain rule to compute $\partial L/\partial W$ for every weight, enabling gradient descent to update weights in the direction that reduces loss.

Q4. Answer: C

Sigmoid outputs $\sigma(x) \in (0,1)$. Tanh outputs $(-1,1)$; ReLU outputs $[0,\infty)$.

Q5. Answer: B

The bias b shifts the neuron's activation threshold independently of the input values, giving the network extra flexibility to fit patterns.

Q6. Answer: C

With 5 classes we need 5 output neurons—one per class. A Softmax activation converts their raw scores into a probability distribution summing to 1.

Q7. Answer: B

In deep networks, gradients are multiplied through many layers. If individual gradients are <1, the product shrinks exponentially, reaching near-zero in early layers. ReLU mitigates this compared to Sigmoid/Tanh.

Q8. Answer: C

Adam (Adaptive Moment Estimation) maintains per-parameter running averages of gradients and squared gradients, adjusting each weight's learning rate individually.

Q9. Answer: C

CNNs use convolutional filters that share weights across spatial locations, making them highly efficient and translation-invariant for image data.

Q10. Answer: B

The learning rate η scales the gradient before subtracting it from the weight: $W \leftarrow W - \eta \cdot \nabla W$. Too large: overshooting; too small: very slow convergence.

Q11. Answer: B

Cross-entropy (binary: $-[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$) is the standard for classification because it directly optimises the log-likelihood of the predicted probability.

Q12. Answer: B

During a forward pass, each layer computes z = Wx+b then a=f(z) until the output layer produces a prediction y_hat. No weights are modified.

Q13. Answer: B

An epoch is one complete pass through all training samples. Models are typically trained for tens to hundreds of epochs until the validation loss stops improving.

Q14. Answer: C

ReLU(x) = max(0, x). For x = –5, max(0, –5) = 0. The neuron outputs zero for any negative input.

Q15. Answer: B

Deep learning's defining feature is its ability to automatically learn hierarchical representations from raw data through many layers—removing the need for manual feature engineering.

Section 10: Glossary of Key Terms

Activation Function	A mathematical function applied to a neuron's weighted sum (z) to introduce non-linearity. Common examples: ReLU, Sigmoid, Tanh. Without activation functions, deep networks would behave like shallow linear models.
Backpropagation	The algorithm for computing the gradient of the loss function with respect to every weight in the network. It applies the chain rule of calculus layer-by-layer from output to input, enabling efficient gradient computation.
Bias (b)	A learnable scalar added to the weighted sum of a neuron (z = Wx + b). It shifts the activation function, allowing the network to fit patterns that don't pass through the origin.
Epoch	One complete forward and backward pass through the entire training dataset. Training typically runs for many epochs. Monitoring validation loss per epoch helps detect overfitting.
Gradient Descent	An optimisation algorithm that updates weights by moving in the direction of the negative gradient of the loss: $W \leftarrow W - \eta \cdot \partial L/\partial W$. Variants include SGD, Mini-batch GD, Adam, and RMSProp.
Hidden Layer	Any layer between the input and output layers. Hidden layers extract increasingly abstract features. The number and size of hidden layers are key hyperparameters.

Hyperparameter	A configuration set before training (not learned from data). Examples: learning rate, number of layers, number of neurons per layer, batch size, and choice of activation function.
Learning Rate (η)	A scalar that controls the step size of weight updates during gradient descent. Too high: oscillation/divergence. Too low: very slow training. Typical values: 0.001–0.1.
Loss Function	A function that quantifies how different the network's predictions are from the true labels. Training minimises this. MSE is used for regression; Cross-Entropy for classification.
Neuron (Node)	The basic computational unit of a neural network. It computes a weighted sum of its inputs, adds a bias, and passes the result through an activation function to produce an output.
Overfitting	When a model learns the training data too specifically and performs poorly on unseen data. Regularisation (dropout, L2) and more data help combat overfitting.
ReLU	Rectified Linear Unit: f(x) = max(0, x). The most widely used hidden-layer activation function. Fast to compute and avoids vanishing gradients for positive inputs.
Sigmoid	Activation function σ(x) = 1/(1+e^–x) that squashes inputs to (0,1). Used in binary classification output layers. Can suffer from vanishing gradients in deep networks.
Softmax	An output-layer activation that converts raw scores (logits) into a probability distribution over K classes. Each output is in (0,1) and all outputs sum to 1.
Tanh	Hyperbolic tangent: f(x) = (e^x – e^–x)/(e^x + e^–x), output in (–1,1). Zero-centred, often preferred over Sigmoid for hidden layers.
Training	The iterative process of adjusting weights and biases by running forward passes, computing loss, running backpropagation, and applying gradient descent. Repeated for many epochs until the model generalises well.
Weight (w)	A learnable scalar parameter that scales the importance of a connection between two neurons. Weights are initialised (e.g., randomly) and updated during training via gradient descent.